

CI/CD in GitLab – Git Repository

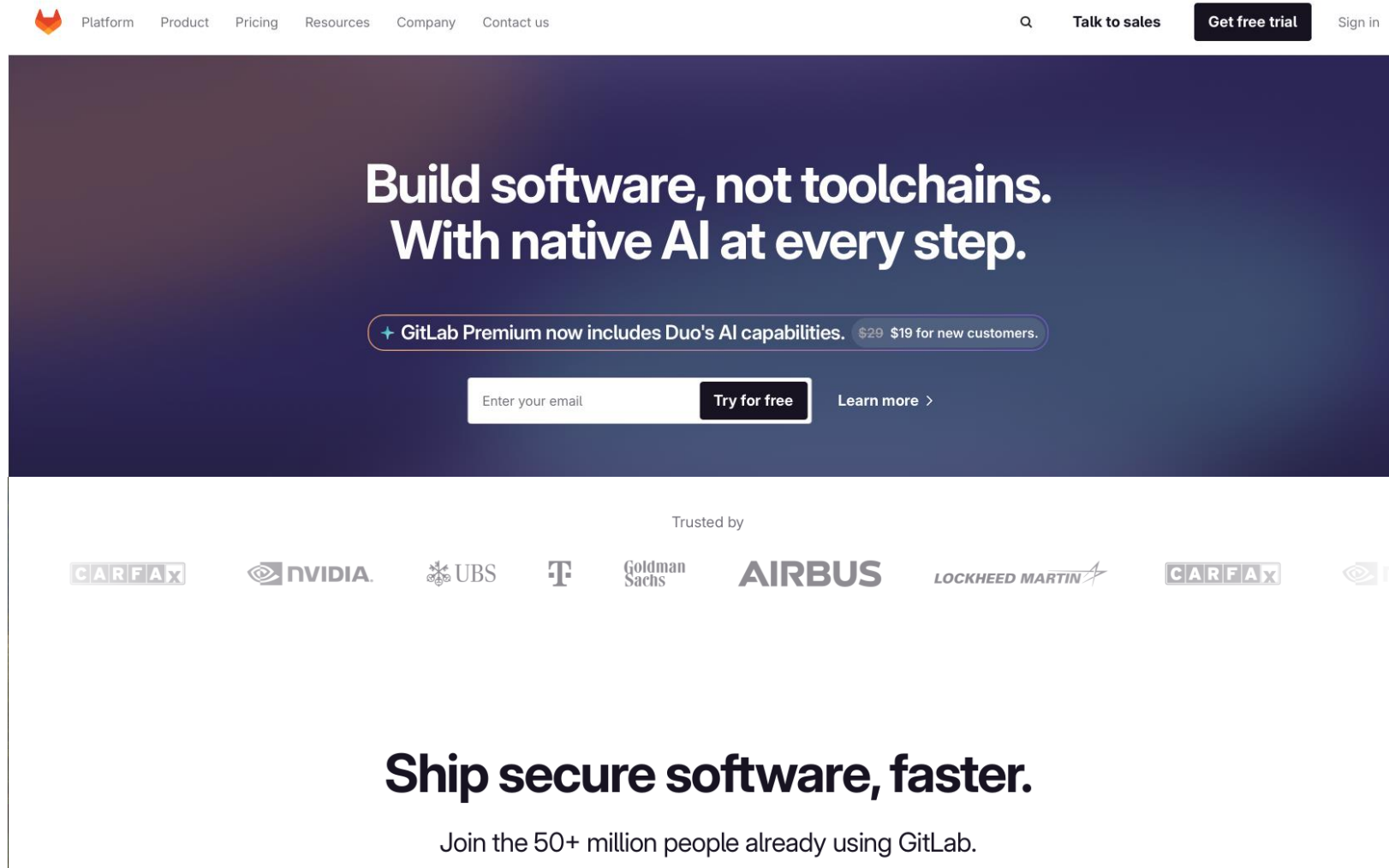
Practical Session 1

Juan Ramírez
jramirezg@faculty.ie.edu

CI/CD with Gitlab

About GitLab

- <https://about.gitlab.com/>



The screenshot shows the GitLab website homepage. At the top, there is a navigation bar with the GitLab logo, links for Platform, Product, Pricing, Resources, Company, and Contact us, a search icon, a 'Talk to sales' link, a 'Get free trial' button, and a 'Sign in' link. The main hero section has a dark blue background with the text 'Build software, not toolchains. With native AI at every step.' Below this, a callout box states '+ GitLab Premium now includes Duo's AI capabilities. \$29 \$19 for new customers.' There is a form to 'Enter your email' with a 'Try for free' button and a 'Learn more >' link. Below the hero section, a 'Trusted by' section displays logos for CARFAX, NVIDIA, UBS, Goldman Sachs, AIRBUS, LOCKHEED MARTIN, and another CARFAX logo. At the bottom, the text 'Ship secure software, faster.' is followed by 'Join the 50+ million people already using GitLab.'

Platform Product Pricing Resources Company Contact us

Q Talk to sales Get free trial Sign in

Build software, not toolchains.
With native AI at every step.

+ GitLab Premium now includes Duo's AI capabilities. \$29 \$19 for new customers.

Enter your email Try for free Learn more >

Trusted by

CARFAX NVIDIA UBS Goldman Sachs AIRBUS LOCKHEED MARTIN CARFAX

Ship secure software, faster.

Join the 50+ million people already using GitLab.

GitLab CI/CD

- Automation Engine for SW development process
- Enables teams to perform DevOps practices
 - Continuous Integration
 - Continuous Delivery/Deployment
- Comparable to Jenkins (wait for later session)
- Automatically build, test and deploy using GitLab pipeline



Work more efficiently

Identify blockers and address them immediately, in a single tool.

[Learn more →](#)



Deliver better software faster

Focus on delivering value—not maintaining integrations.

[Learn more →](#)



Reduce risk and cost

Automate security and compliance without compromising speed or spend.

[Learn more →](#)

GitLab Pipelines

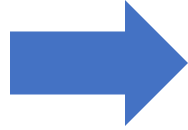
- `.gitlab-ci.yml`



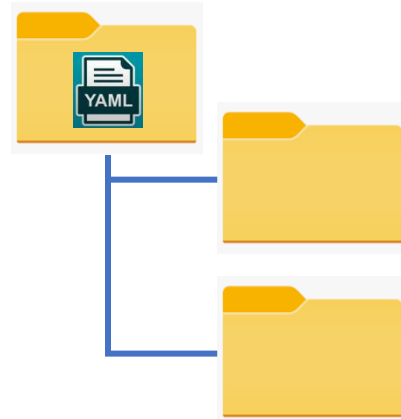
Configuration file that lives in the root of a GitLab project



Automated set of sequential steps to build, test and deliver/deploy your code



YAML is a human-readable data-serialization language. It is commonly used for configuration files and in applications where data is being stored or transmitted



GitLab Pipelines (cont.)

- Two main components:
 - **Jobs** -> Describe the tasks that need to be done
 - Could include commands to compile, test or deploy code
 - **Stages** -> Define the order in which jobs will be completed
- Pipeline -> Set of instructions for a program to execute
- The program that execute Jobs in a GitLab project is called Gitlab Runner

Create Blank Project



Create new project



Create blank project

Create a blank project to store your files, plan your work, and collaborate on code, among other things.



Create from template

Create a project pre-populated with the necessary files to get you started quickly.



Import project

Migrate your data from an external source like GitHub, Bitbucket, or another instance of GitLab.

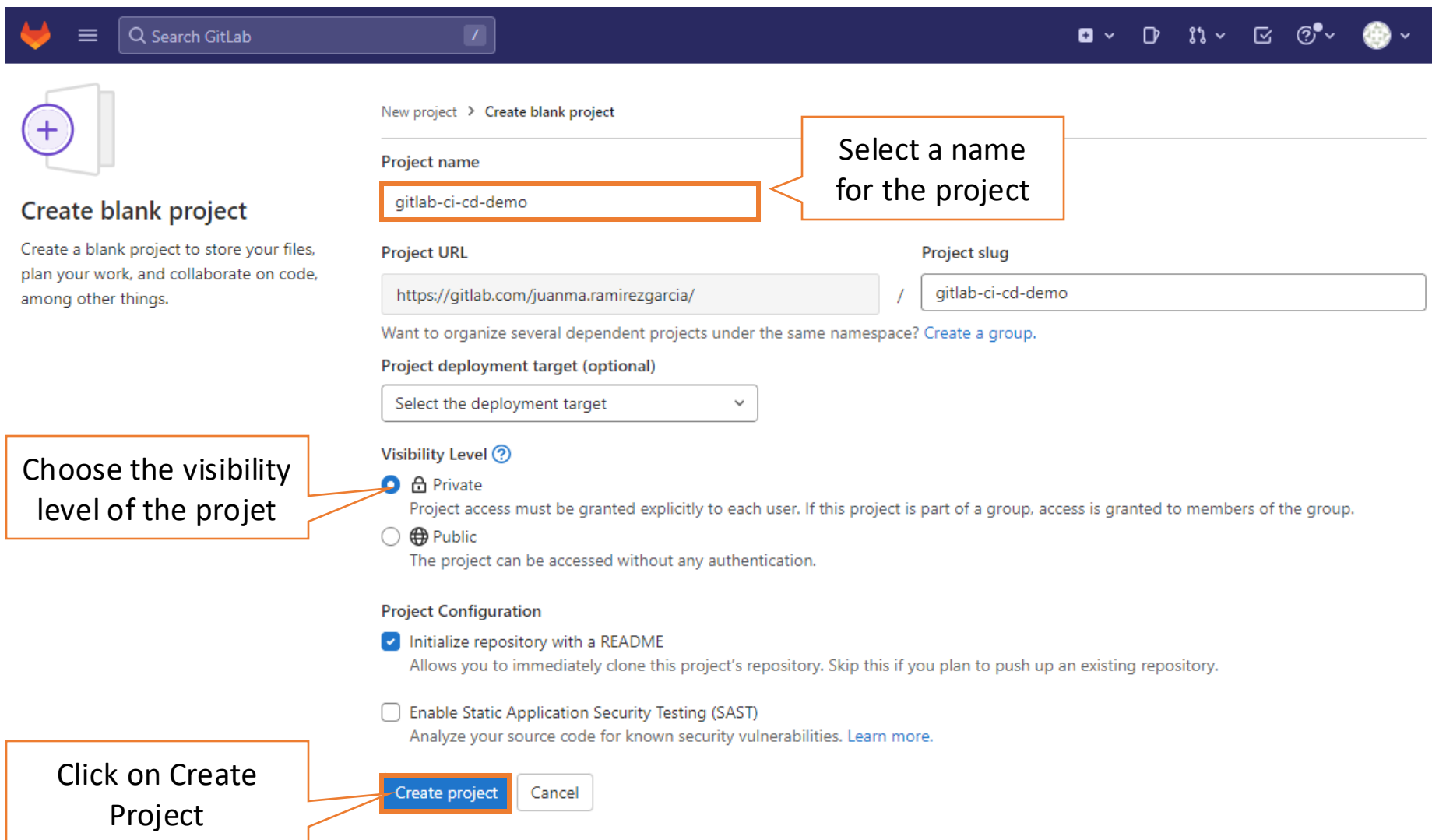


Run CI/CD for external repository

Connect your external repository to GitLab CI/CD.

You can also create a project from the command line. [Show command](#)

Fill Project Name



The image shows the 'Create blank project' form in GitLab. The form is titled 'New project > Create blank project'. It includes a 'Project name' field with the value 'gitlab-ci-cd-demo', a 'Project URL' field with the value 'https://gitlab.com/juanma.ramirezgarcia/', and a 'Project slug' field with the value 'gitlab-ci-cd-demo'. There is a 'Project deployment target (optional)' dropdown menu with the value 'Select the deployment target'. The 'Visibility Level' section has two radio buttons: 'Private' (selected) and 'Public'. The 'Project Configuration' section has two checkboxes: 'Initialize repository with a README' (checked) and 'Enable Static Application Security Testing (SAST)' (unchecked). The 'Create project' button is highlighted with an orange box.

Create blank project

Create a blank project to store your files, plan your work, and collaborate on code, among other things.

New project > Create blank project

Project name

gitlab-ci-cd-demo

Project URL

https://gitlab.com/juanma.ramirezgarcia/

Project slug

gitlab-ci-cd-demo

Want to organize several dependent projects under the same namespace? [Create a group.](#)

Project deployment target (optional)

Select the deployment target

Visibility Level ?

☒ Private
Project access must be granted explicitly to each user. If this project is part of a group, access is granted to members of the group.

☐ Public
The project can be accessed without any authentication.

Project Configuration

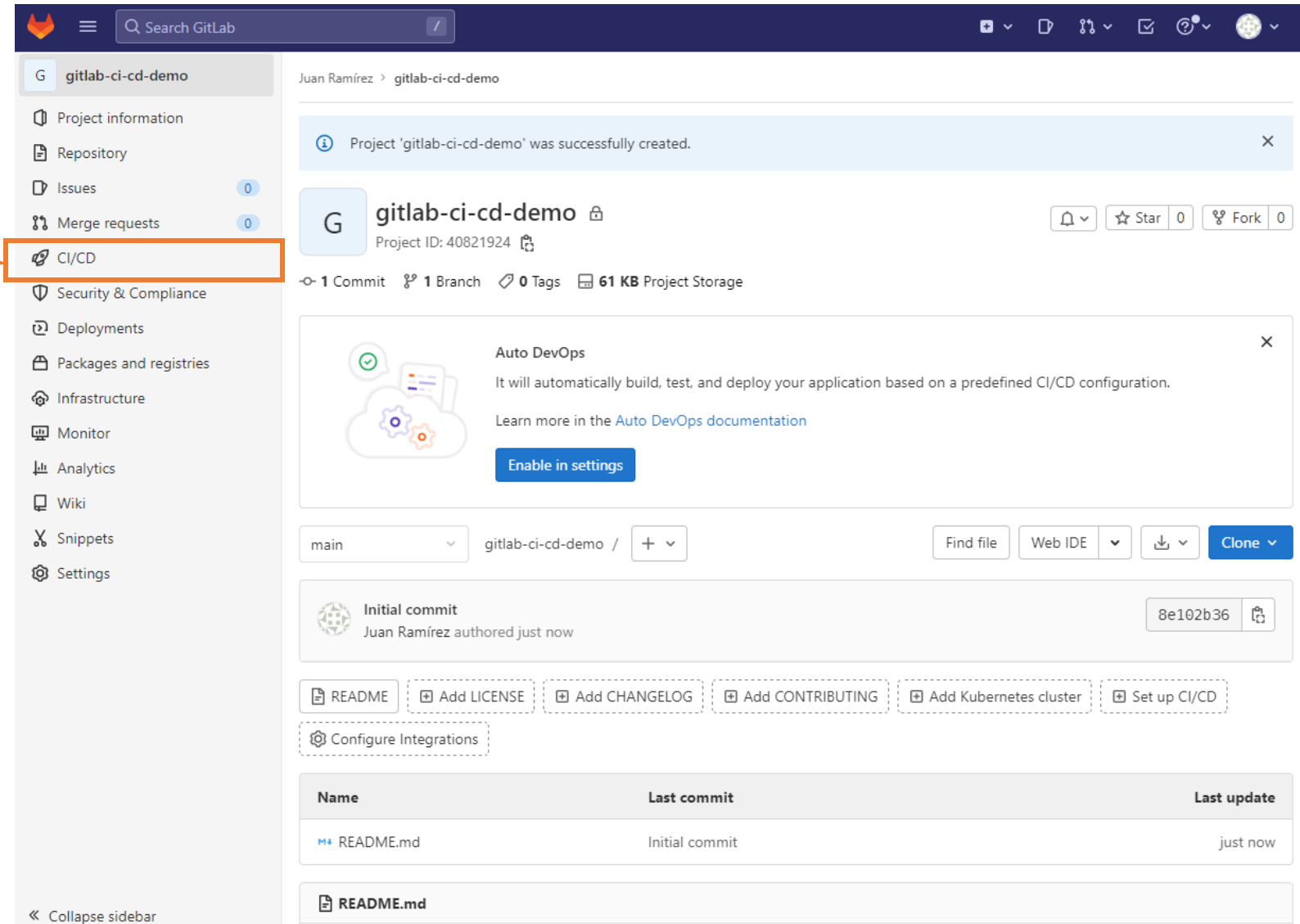
☒ Initialize repository with a README
Allows you to immediately clone this project's repository. Skip this if you plan to push up an existing repository.

☐ Enable Static Application Security Testing (SAST)
Analyze your source code for known security vulnerabilities. [Learn more.](#)

Create project Cancel

New Project

Click on the
CI/CD section



Search GitLab

gitlab-ci-cd-demo

- Project information
- Repository
- Issues 0
- Merge requests 0
- CI/CD**
- Security & Compliance
- Deployments
- Packages and registries
- Infrastructure
- Monitor
- Analytics
- Wiki
- Snippets
- Settings

Juan Ramírez > gitlab-ci-cd-demo

Project 'gitlab-ci-cd-demo' was successfully created.

gitlab-ci-cd-demo Project ID: 40821924

1 Commit 1 Branch 0 Tags 61 KB Project Storage

Auto DevOps
It will automatically build, test, and deploy your application based on a predefined CI/CD configuration.
Learn more in the [Auto DevOps documentation](#)
[Enable in settings](#)

main gitlab-ci-cd-demo / + Find file Web IDE Clone

Initial commit
Juan Ramírez authored just now 8e102b36

[README](#) [Add LICENSE](#) [Add CHANGELOG](#) [Add CONTRIBUTING](#) [Add Kubernetes cluster](#) [Set up CI/CD](#)

[Configure Integrations](#)

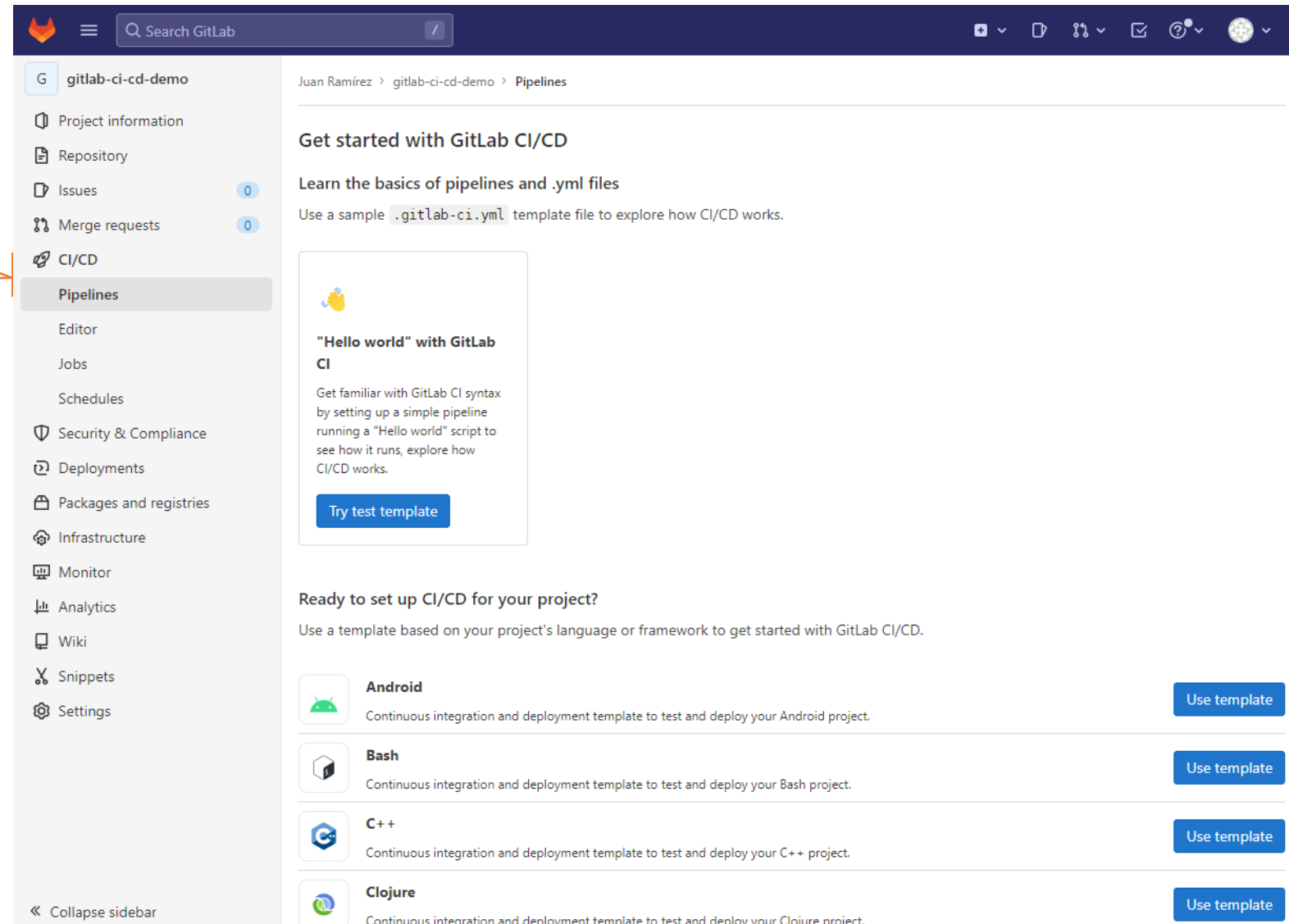
Name	Last commit	Last update
README.md	Initial commit	just now

[README.md](#)

« Collapse sidebar

CI/CD

Click on the
CI/CD section



The screenshot shows the GitLab web interface for a project named 'gitlab-ci-cd-demo'. The left sidebar contains a navigation menu with options: Project information, Repository, Issues (0), Merge requests (0), CI/CD, Pipelines (selected), Editor, Jobs, Schedules, Security & Compliance, Deployments, Packages and registries, Infrastructure, Monitor, Analytics, Wiki, Snippets, and Settings. The main content area is titled 'Pipelines' and includes a search bar, a breadcrumb trail 'Juan Ramírez > gitlab-ci-cd-demo > Pipelines', and a 'Get started with GitLab CI/CD' section. This section contains a 'Hello world' with GitLab CI tutorial card with a 'Try test template' button. Below this is a 'Ready to set up CI/CD for your project?' section with a list of templates for Android, Bash, C++, and Clojure, each with a 'Use template' button.

Search GitLab

gitlab-ci-cd-demo

- Project information
- Repository
- Issues 0
- Merge requests 0
- CI/CD
- Pipelines**
- Editor
- Jobs
- Schedules
- Security & Compliance
- Deployments
- Packages and registries
- Infrastructure
- Monitor
- Analytics
- Wiki
- Snippets
- Settings

« Collapse sidebar

Juan Ramírez > gitlab-ci-cd-demo > Pipelines

Get started with GitLab CI/CD

Learn the basics of pipelines and .yml files

Use a sample `.gitlab-ci.yml` template file to explore how CI/CD works.



"Hello world" with GitLab CI

Get familiar with GitLab CI syntax by setting up a simple pipeline running a "Hello world" script to see how it runs, explore how CI/CD works.

[Try test template](#)

Ready to set up CI/CD for your project?

Use a template based on your project's language or framework to get started with GitLab CI/CD.

	Android Continuous integration and deployment template to test and deploy your Android project.	Use template
	Bash Continuous integration and deployment template to test and deploy your Bash project.	Use template
	C++ Continuous integration and deployment template to test and deploy your C++ project.	Use template
	Clojure Continuous integration and deployment template to test and deploy your Clojure project.	Use template

Configure Pipeline

Select Editor and click on the
'Configure Pipeline' button

g gitlab-ci-cd-demo

- Project information
- Repository
- Issues 0
- Merge requests 0
- CI/CD
- Pipelines
- Editor**
- Jobs
- Schedules
- Security & Compliance
- Deployments
- Packages and registries
- Infrastructure
- Monitor
- Analytics
- Wiki
- Snippets
- Settings

Juan Ramírez > gitlab-ci-cd-demo > Pipeline Editor

main

Optimize your workflow with CI/CD Pipelines

Create a new `.gitlab-ci.yml` file at the root of the repository to get started.

Configure pipeline

Configure Pipeline (cont.)

GitLab shows a pipeline example with Stages and Jobs

The screenshot displays the GitLab Pipeline Editor for the project 'gitlab-ci-cd-demo'. The left sidebar contains navigation links: Project information, Repository, Issues (0), Merge requests (0), CI/CD, Pipelines, Editor, Deployments, Packages and registries, Infrastructure, Monitor, and Settings. The main editor area shows the 'Pipeline Editor' for the 'main' branch. A tooltip explains that the '.gitlab-ci.yml' file creates a simple test pipeline and instructs the user to use the 'commit changes' button to run the pipeline. The YAML file content is as follows:

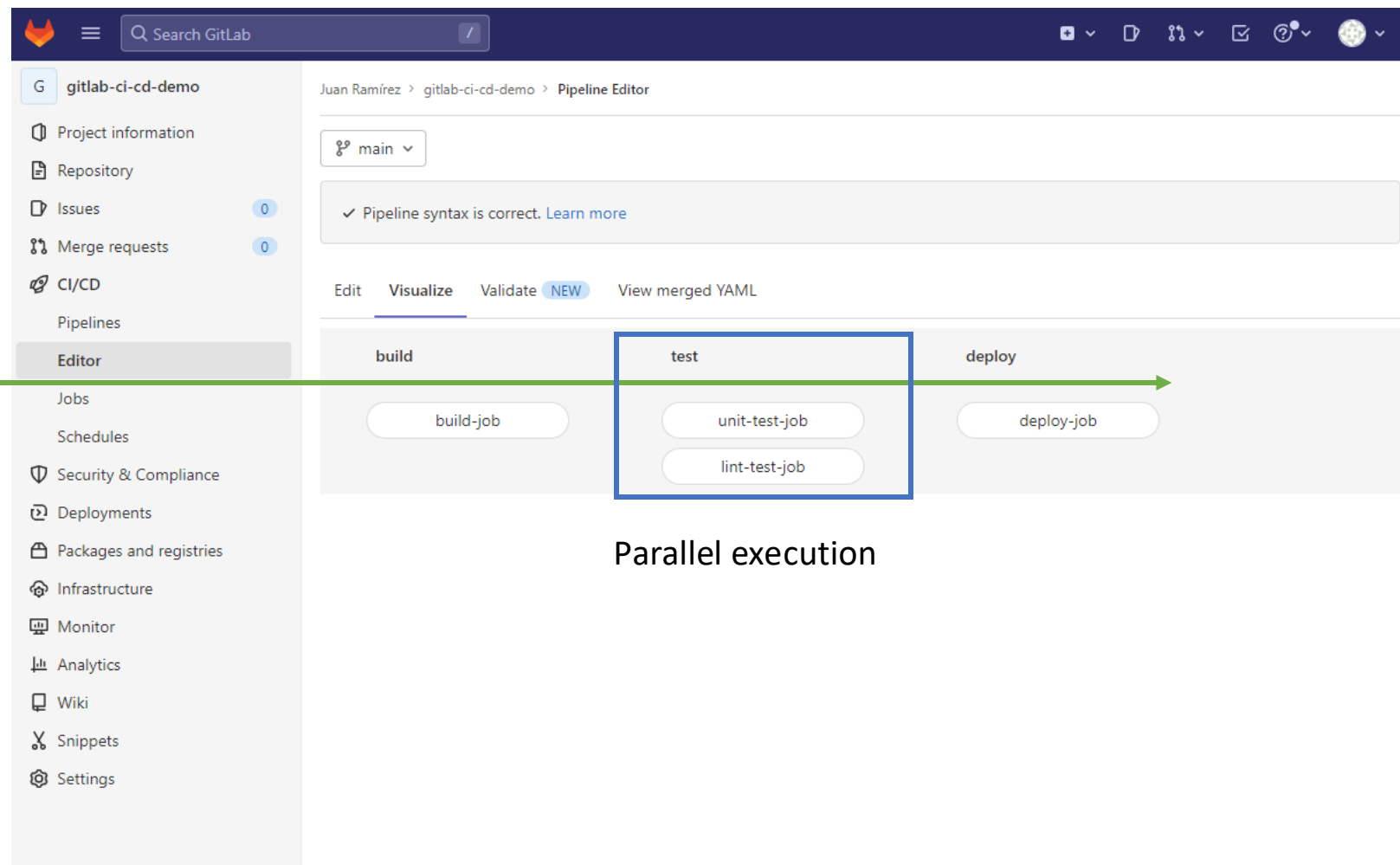
```
1 # https://gitlab.com/gitlab-org/gitlab/-/blob/master/Lib/gitlab/ci/templates/Getting-Started.gitlab-ci.yml
18
19 stages: # List of stages for jobs, and their order of execution
20   - build
21   - test
22   - deploy
23
24 build-job: # This job runs in the build stage, which runs first.
25   stage: build
26   script:
27     - echo "Compiling the code..."
28     - echo "Compile complete."
29
30 unit-test-job: # This job runs in the test stage.
31   stage: test # It only starts when the job in the build stage completes successfully.
32   script:
33     - echo "Running unit tests... This will take about 60 seconds."
34     - sleep 60
35     - echo "Code coverage is 90%"
36
37 lint-test-job: # This job also runs in the test stage.
38   stage: test # It can run at the same time as unit-test-job (in parallel).
39   script:
40     - echo "Linting code... This will take about 10 seconds."
41     - sleep 10
42     - echo "No lint issues found."
```

At the bottom, the 'Commit message' field contains 'Update .gitlab-ci.yml file'.

Configure Pipeline (cont.)

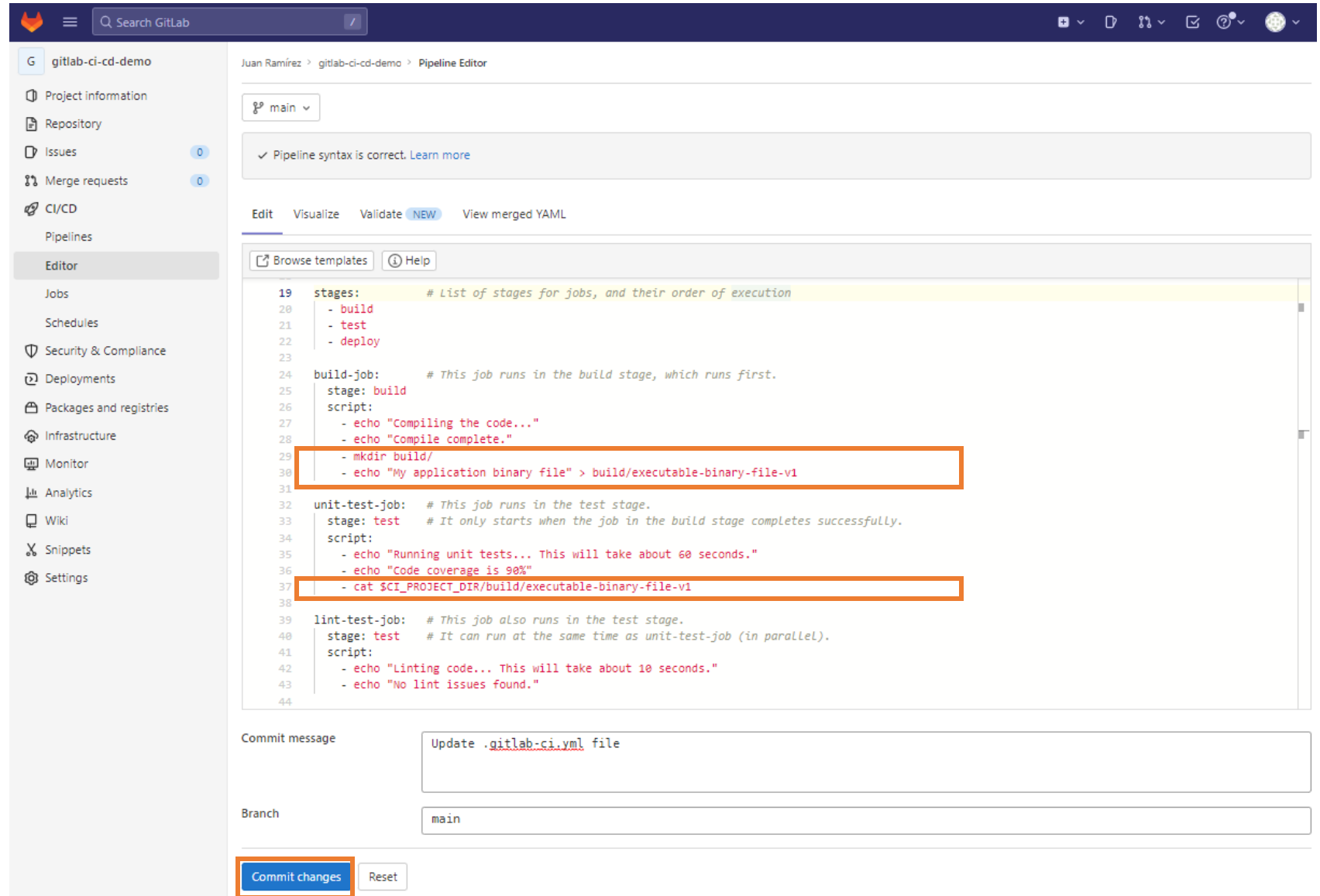
Visualize tab presents a graphical view of stages and jobs

Order of execution



Edit and Execute Pipeline

Do some simple edits and then
'Commit changes' to start executing
the pipeline



GitLab Pipeline Editor interface showing the configuration for the 'main' branch. The pipeline is defined in the 'build' stage, which includes a 'build-job' that runs a script to compile the code and create a binary file. The 'test' stage includes two jobs: 'unit-test-job' and 'lint-test-job'.

```
19 stages: # List of stages for jobs, and their order of execution
20 - build
21 - test
22 - deploy
23
24 build-job: # This job runs in the build stage, which runs first.
25   stage: build
26   script:
27     - echo "Compiling the code..."
28     - echo "compile complete."
29     - mkdir build/
30     - echo "My application binary file" > build/executable-binary-file-v1
31
32 unit-test-job: # This job runs in the test stage.
33   stage: test # It only starts when the job in the build stage completes successfully.
34   script:
35     - echo "Running unit tests... This will take about 60 seconds."
36     - echo "Code coverage is 90%"
37     - cat $CI_PROJECT_DIR/build/executable-binary-file-v1
38
39 lint-test-job: # This job also runs in the test stage.
40   stage: test # It can run at the same time as unit-test-job (in parallel).
41   script:
42     - echo "Linting code... This will take about 10 seconds."
43     - echo "No lint issues found."
44
```

Commit message: Update `.gitlab-ci.yml` file

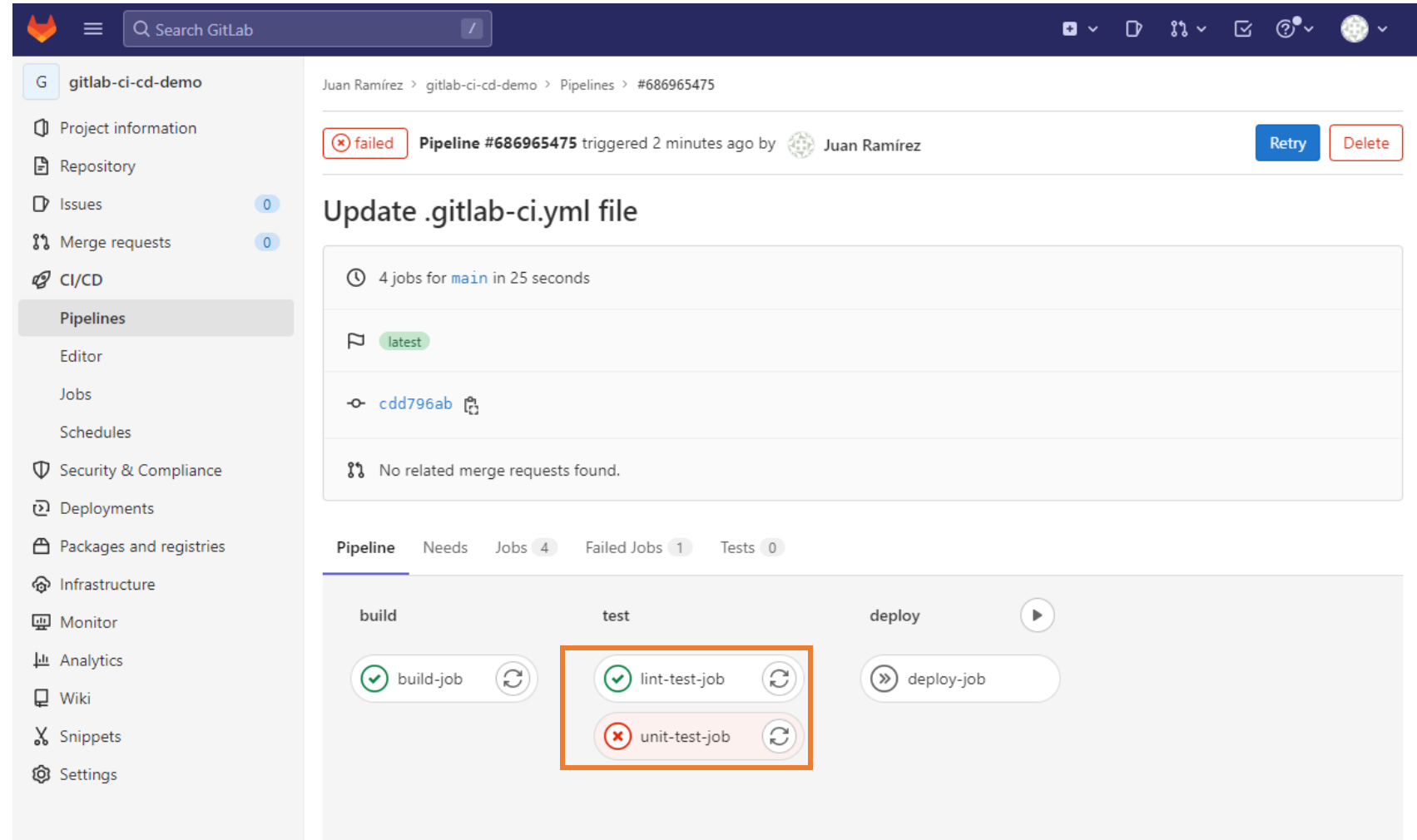
Branch: main

Commit changes Reset

First Run Fails

There is one failure in the 'unit-test-job'. Why did this happen?

- The jobs are executed in different containers
- We need a way to share artifacts between different jobs

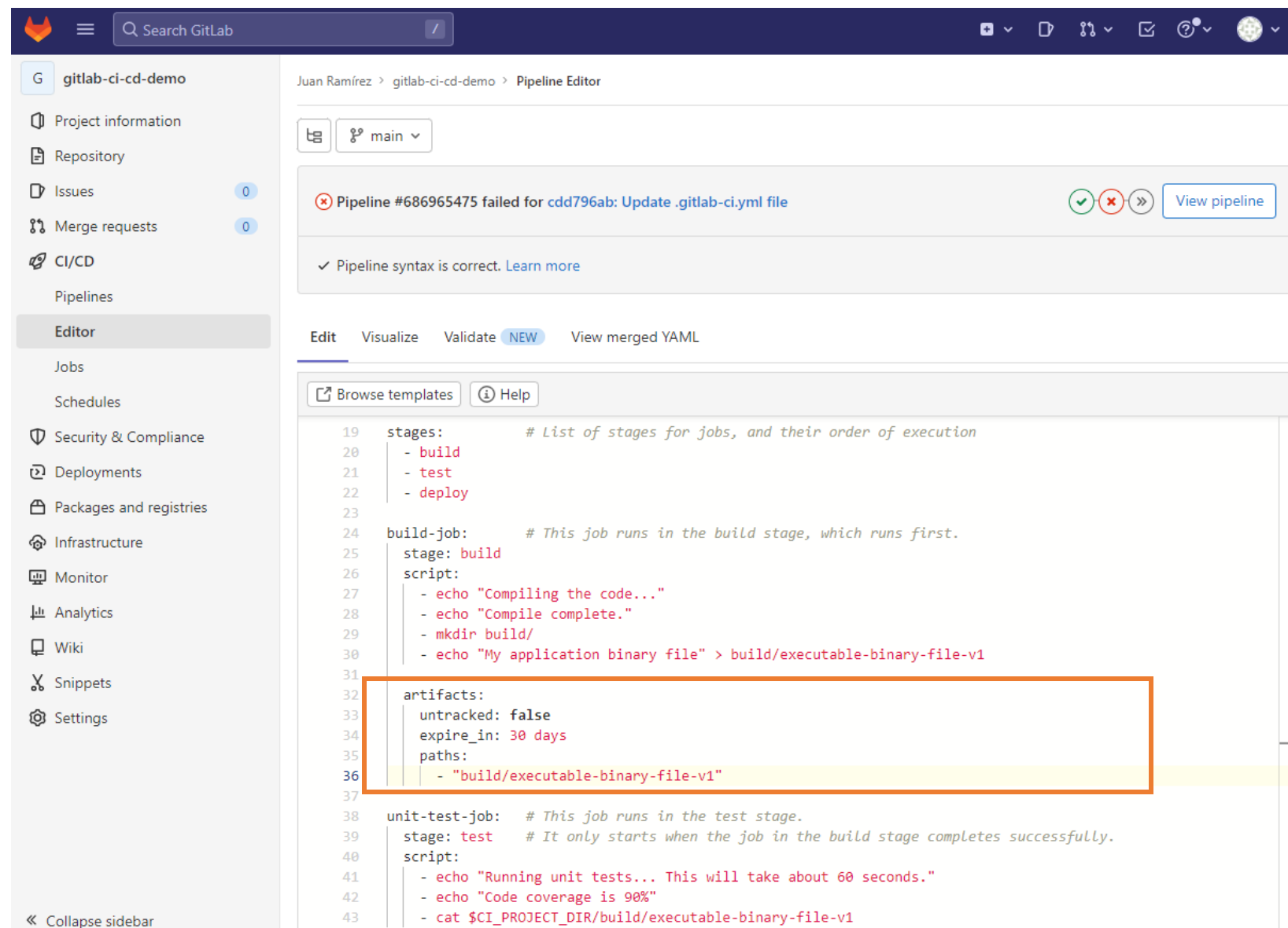


The screenshot shows the GitLab CI/CD interface for a project named 'gitlab-ci-cd-demo'. The pipeline #686965475 is shown as failed. The pipeline consists of four jobs: 'build', 'test', and 'deploy'. The 'test' stage contains two jobs: 'lint-test-job' and 'unit-test-job'. The 'unit-test-job' is highlighted with a red box and a red 'X' icon, indicating it failed. The 'build' stage has a 'build-job' with a green checkmark. The 'deploy' stage has a 'deploy-job' with a green checkmark. The 'test' stage has a 'lint-test-job' with a green checkmark. The pipeline summary shows 4 jobs for 'main' in 25 seconds. The 'unit-test-job' failure is the only one in the pipeline.

GitLab CI/CD interface showing a failed pipeline (#686965475) for the project 'gitlab-ci-cd-demo'. The pipeline status is 'failed' (indicated by a red 'X' icon). The pipeline was triggered 2 minutes ago by Juan Ramírez. The pipeline summary shows 4 jobs for 'main' in 25 seconds. The jobs are: build (build-job), test (lint-test-job, unit-test-job), and deploy (deploy-job). The 'unit-test-job' is highlighted with a red box and a red 'X' icon, indicating it failed. The 'build-job' and 'lint-test-job' are marked with green checkmarks, indicating they passed. The 'deploy-job' is marked with a green checkmark, indicating it passed. The pipeline summary also shows 'No related merge requests found.'

Artifacts

Edit 'artifacts' section in the build-job



GitLab Pipeline Editor interface for the project 'gitlab-ci-cd-demo'.

The interface shows the Pipeline Editor for the 'main' branch. A message indicates that Pipeline #686965475 failed for commit cdd796ab: Update .gitlab-ci.yml file. The Pipeline syntax is correct.

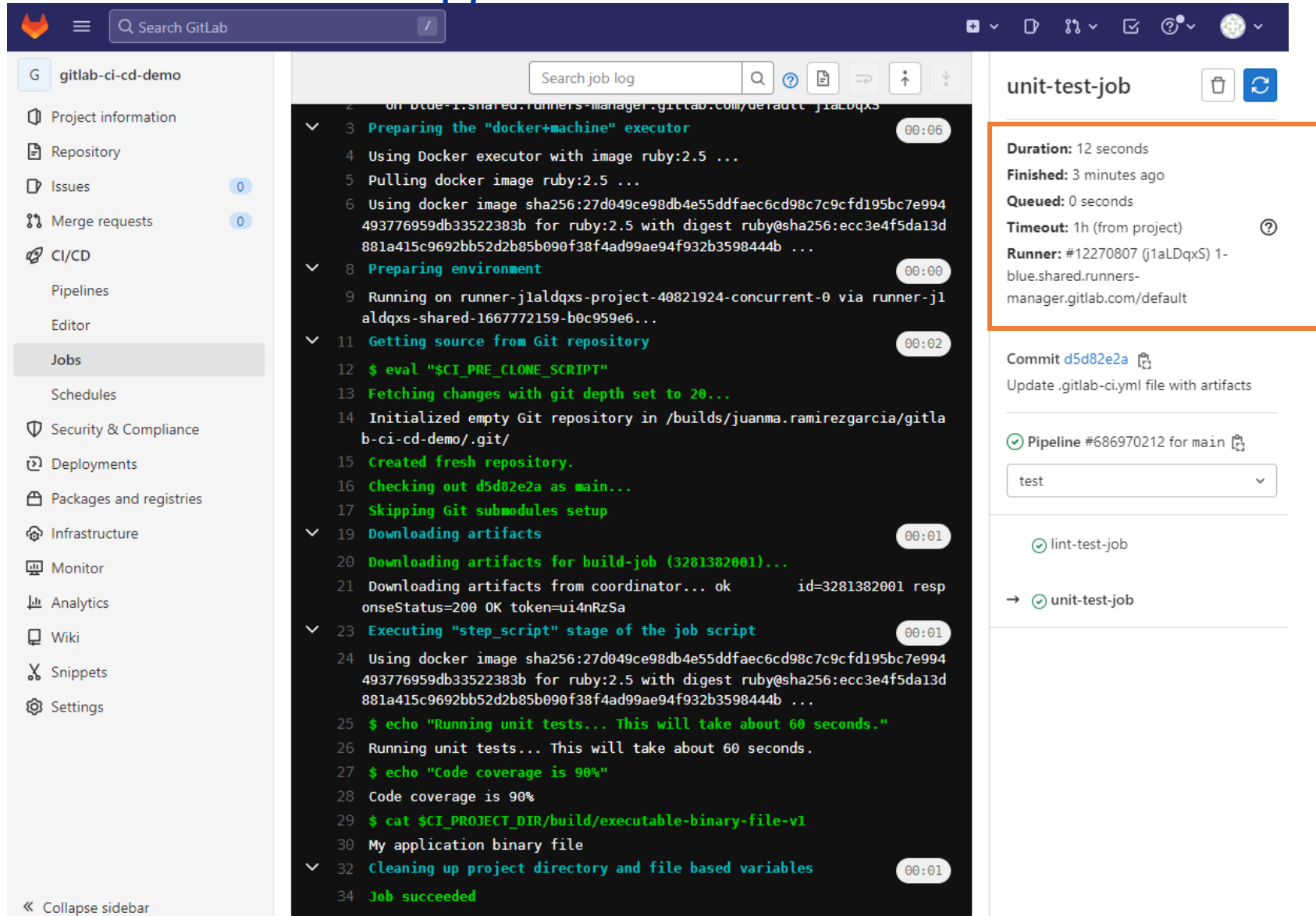
The editor shows the following configuration for the 'build-job' stage:

```
stages:
  - build
  - test
  - deploy

build-job:
  stage: build
  script:
    - echo "Compiling the code..."
    - echo "Compile complete."
    - mkdir build/
    - echo "My application binary file" > build/executable-binary-file-v1

artifacts:
  untracked: false
  expire_in: 30 days
  paths:
    - "build/executable-binary-file-v1"
```

Pipeline Execution Log



The screenshot displays the GitLab CI/CD interface for a project named 'gitlab-ci-cd-demo'. The left sidebar contains navigation links for Project information, Repository, Issues (0), Merge requests (0), CI/CD (selected), Pipelines, Editor, Jobs, Schedules, Security & Compliance, Deployments, Packages and registries, Infrastructure, Monitor, Analytics, Wiki, Snippets, and Settings. The main area shows the 'unit-test-job' execution log. The log is divided into sections: 'Preparing the "docker+machine" executor' (00:06), 'Preparing environment' (00:00), 'Getting source from Git repository' (00:02), 'Downloading artifacts' (00:01), 'Executing "step_script" stage of the job script' (00:01), and 'Cleaning up project directory and file based variables' (00:01). The log text includes commands like 'Using Docker executor with image ruby:2.5 ...', 'Pulling docker image ruby:2.5 ...', 'Running on runner-j1aldqxs-project-40821924-concurrent-0 via runner-j1aldqxs-shared-1667772159-b0c959e6...', 'Fetching changes with git depth set to 20...', 'Created fresh repository.', 'Checking out d5d82e2a as main...', 'Skipping Git submodules setup', 'Downloading artifacts for build-job (3281382001)...', 'Downloading artifacts from coordinator... ok id=3281382001 responseStatus=200 OK token=ui4nRzSa', 'Using docker image sha256:27d049ce98db4e55ddfaec6cd98c7c9cfd195bc7e994493776959db33522383b for ruby:2.5 with digest ruby@sha256:ecc3e4f5da13d881a415c9692bb52d2b85b090f38f4ad99ae94f932b3598444b ...', '\$ echo "Running unit tests... This will take about 60 seconds."', 'Running unit tests... This will take about 60 seconds.', '\$ echo "Code coverage is 90%"', 'Code coverage is 90%', '\$ cat \$CI_PROJECT_DIR/build/executable-binary-file-v1', 'My application binary file', and 'Job succeeded'. The right sidebar shows job details for 'unit-test-job': Duration: 12 seconds, Finished: 3 minutes ago, Queued: 0 seconds, Timeout: 1h (from project), Runner: #12270807 (j1aLDqXS) 1-blue.shared.runners-manager.gitlab.com/default. Below this, it shows the commit 'd5d82e2a' and the pipeline '#686970212 for main' with a dropdown menu set to 'test'. At the bottom, it shows a list of jobs: 'lint-test-job' and 'unit-test-job' (selected).

Search job log

unit-test-job

Duration: 12 seconds
Finished: 3 minutes ago
Queued: 0 seconds
Timeout: 1h (from project)
Runner: #12270807 (j1aLDqXS) 1-blue.shared.runners-manager.gitlab.com/default

Commit d5d82e2a
Update .gitlab-ci.yml file with artifacts

✓ Pipeline #686970212 for main
test

✓ lint-test-job
→ ✓ unit-test-job

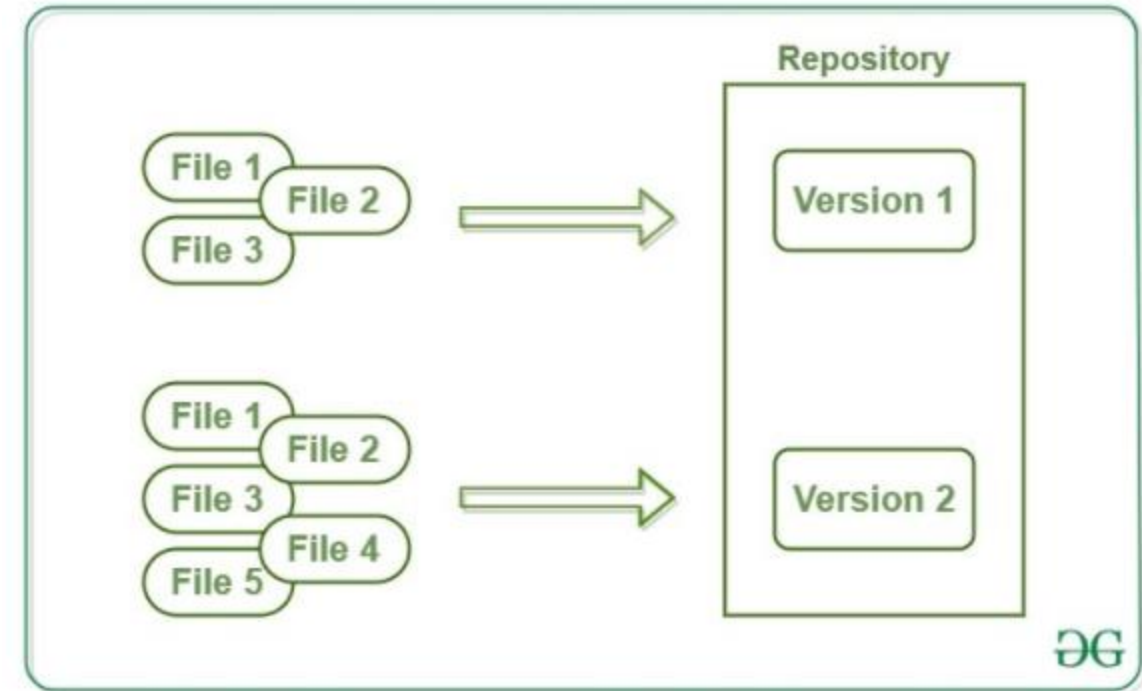
Git Repository

What is a Git

- Git is a Version Control System (VCS) designed to make it easier to have multiple versions of a code base, sometimes across multiple developers or teams
- It allows you to see changes you make to your code and easily revert them

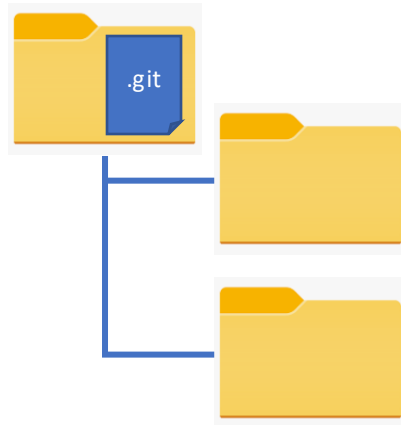
What is a Git Repository?

- Repositories in GIT contain a collection of files of various different versions of a Project.
- These files are imported from the repository into the local server of the user for further updates and modifications in the content of the file.
- A **Version Control System (VCS)** is used to create these versions and store them in a specific place termed as a repository.
- The process of copying the content from an existing Git Repository with the help of various Git Tools is termed as **cloning**.
- Once the cloning process is done, the user gets the complete repository on his local machine. Git by default assumes the work to be done on the repository is as a user, once the cloning is done.



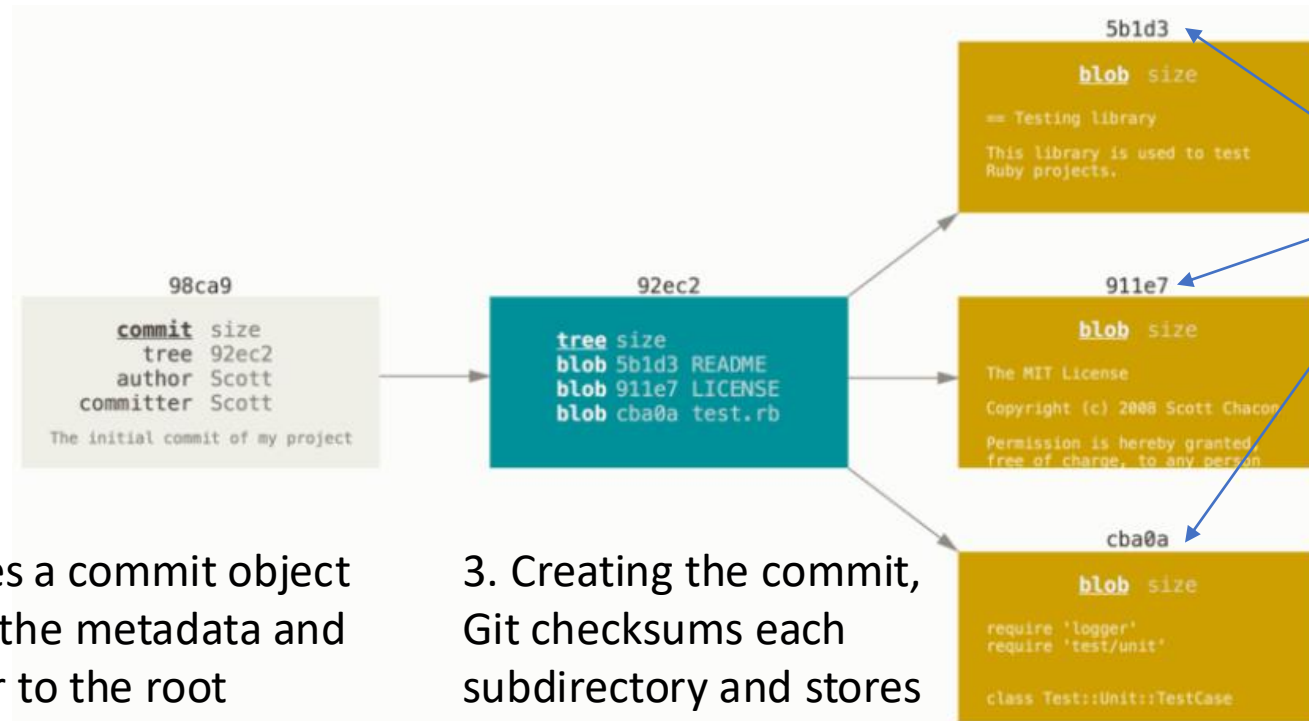
Repository

- A repository is a data structure containing a variety of objects that represent the current status of a git project and its history and versions.
- Repositories can be stored locally or remotely, and they can be synced to each other.
- The repository information in Git is stored in a hidden directory called **.git**



Commit

1. Three files in a directory



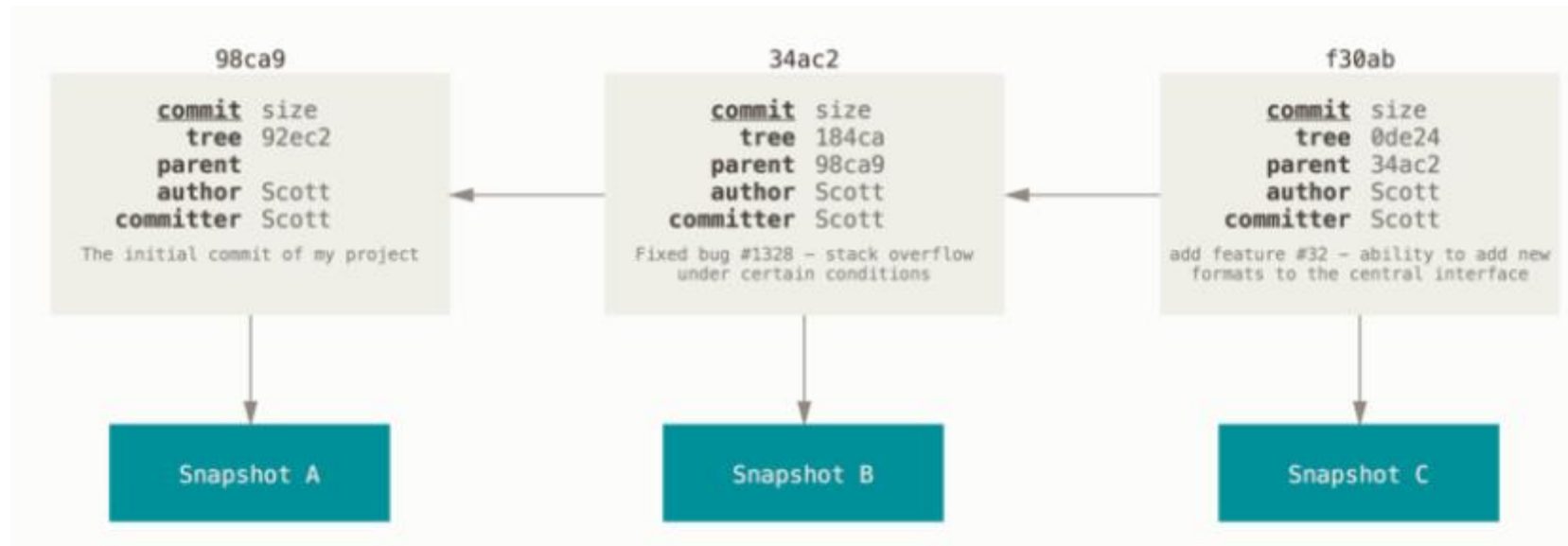
2. Staging the files computes a checksum, stores that version of the file in the Git repository and adds that checksum to the staging area

4. Creates a commit object that has the metadata and a pointer to the root project tree

3. Creating the commit, Git checksums each subdirectory and stores them in a tree object

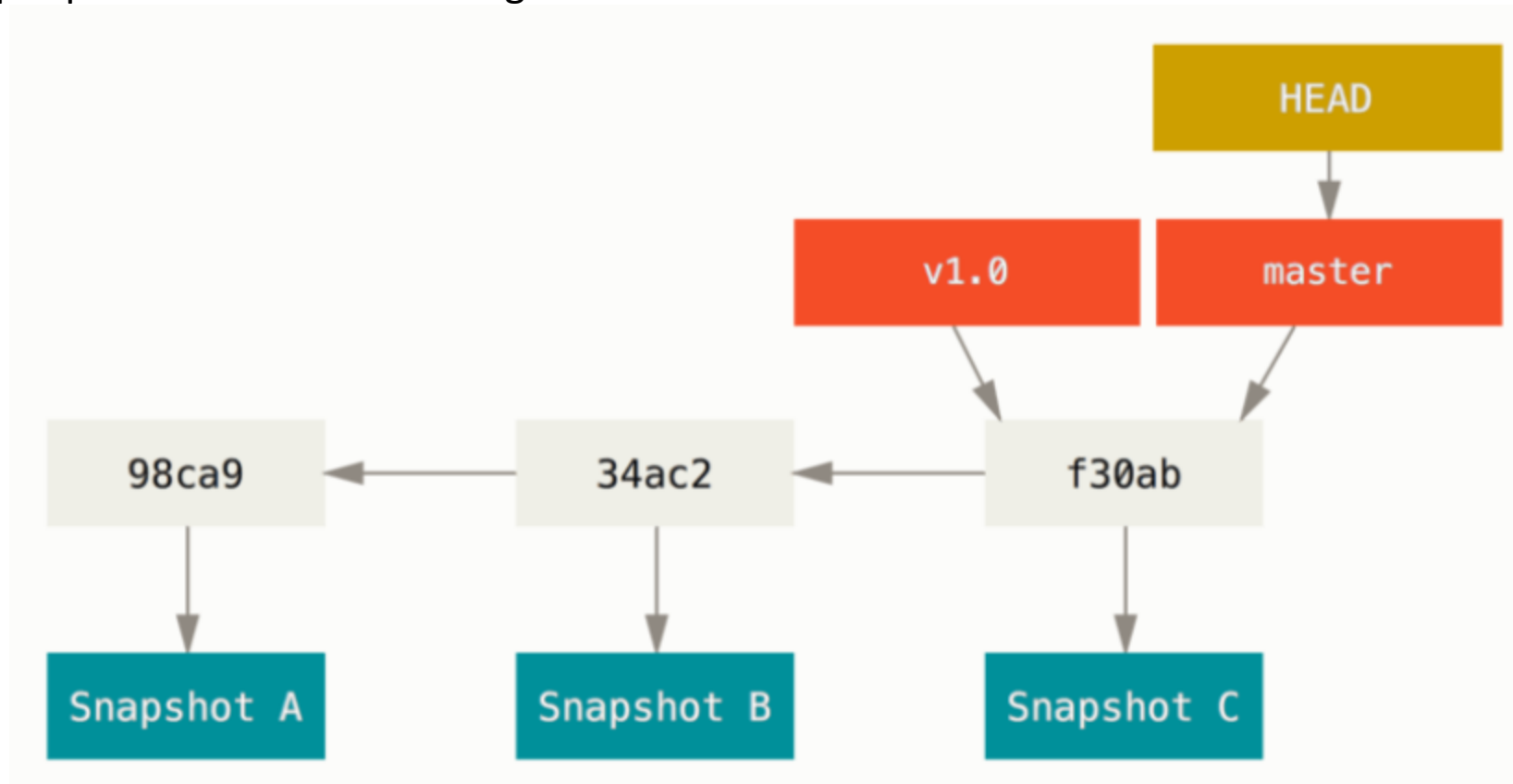
Making changes

If you make some changes and commit again, the next commit stores a pointer to the commit that came immediately before it.

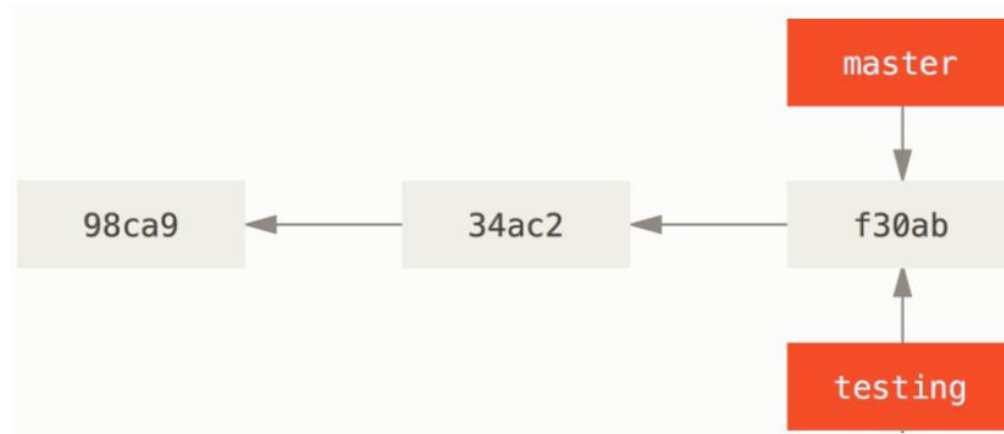


Branch and Commit History

The “master” branch in Git is not a special branch. It is exactly like any other branch. The only reason nearly every repository has one is that the git init command creates it by default and most people don’t bother to change it.

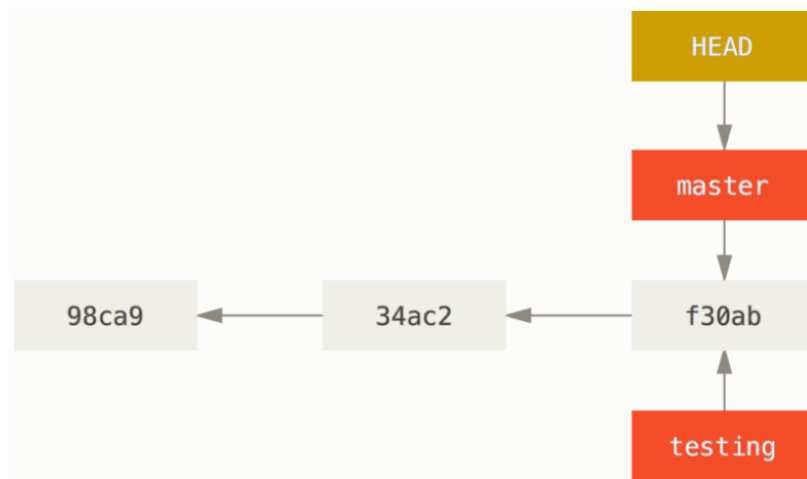


Creating New Branch and Pointing



We create a new branch called 'testing'

Special pointer called HEAD points to the local branch you're currently on

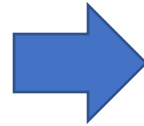
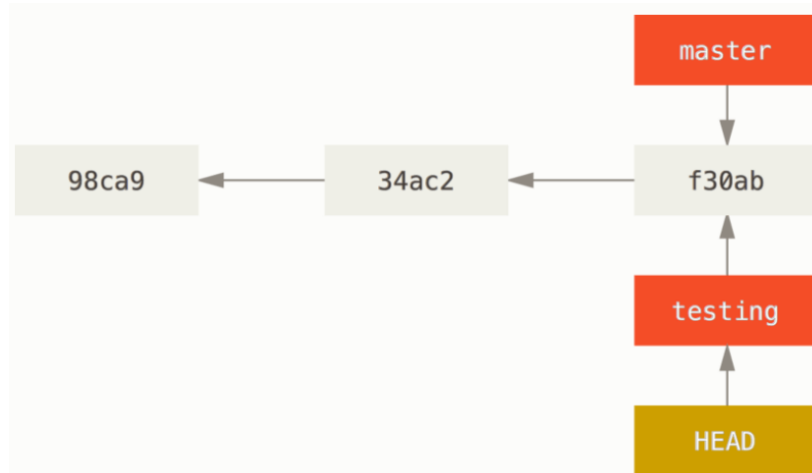


You can switch to point HEAD to the new branch

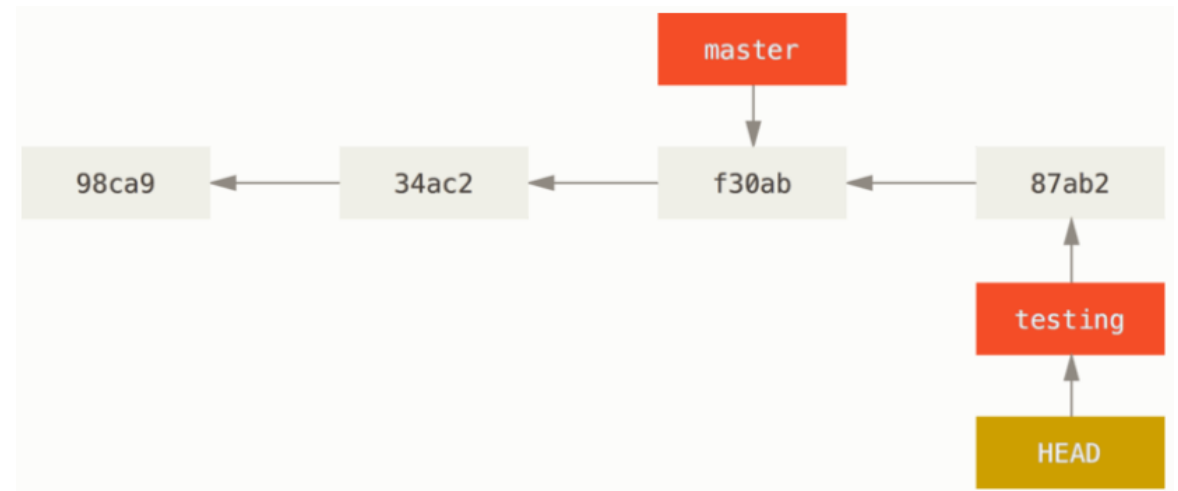


Committing a New Change

After creating the new branch 'testing'

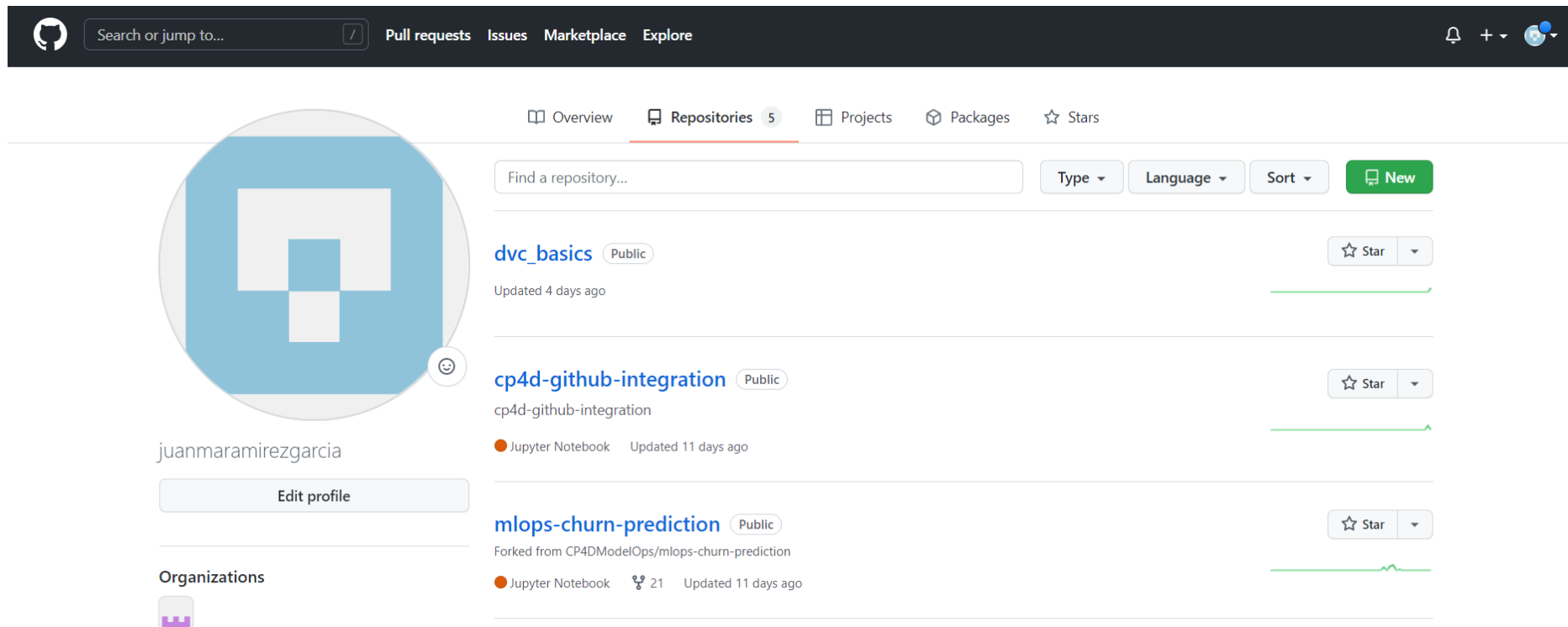


After committing a change



What is GitHub

- Github.com is a website that hosts git repositories on a remote server
- Hosting repositories on GitHub facilitates the sharing of codebases among teams by providing a GUI to easily fork or clone repos to a local machine
- By pushing your repositories to GitHub, you will pretty much automatically create your own developer portfolio as well!



GitHub as Platform for Open Source projects

- Publicly available to use and contribute to the project

About

 CML - Continuous Machine Learning |
CI/CD for ML

 cml.dev

[data-science](#) [machine-learning](#)
[continuous-integration](#) [continuous-delivery](#)
[ci](#) [gitlab-ci](#) [ci-cd](#) [developer-tools](#)
[cicd](#) [dvc](#) [github-actions](#)

 Readme

 Apache-2.0 License

 3.1k stars

 49 watching

 222 forks

How many using

How many contributing

Releases 30

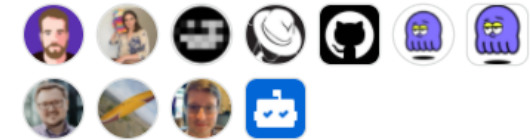
 CML v0.12.1 [Latest](#)
4 days ago

[+ 29 releases](#)

Packages 1

 cml

Contributors 25



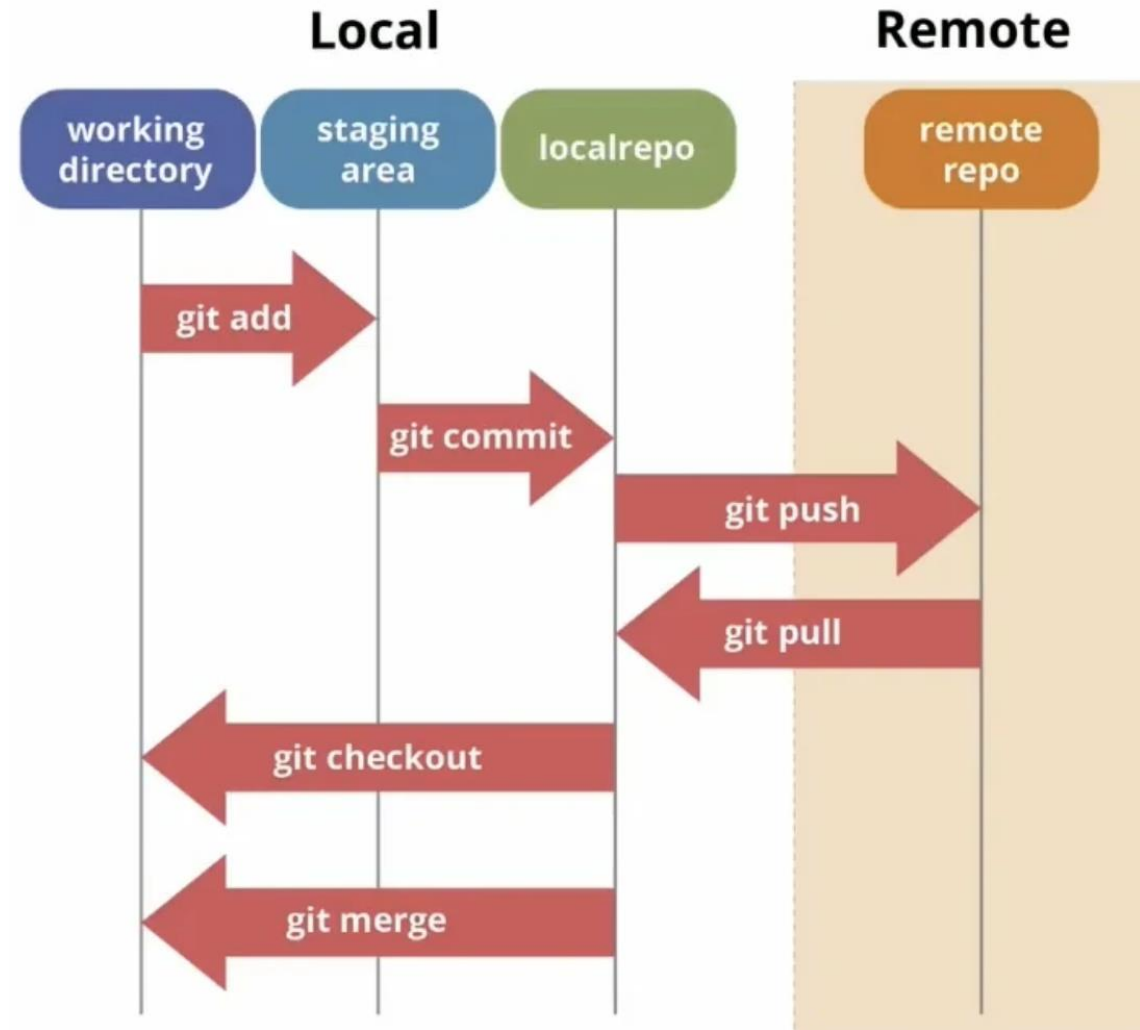
[+ 14 contributors](#)

Environments 2

 internal [Active](#)

 external [Active](#)

Working with a Git Repository Local and Remote



Test Git in your Local Machine

- <https://git-scm.com/>

The screenshot shows the Git website homepage. At the top, the Git logo is followed by the tagline "--distributed-even-if-your-workflow-isnt". A search bar is on the right. The main text describes Git as a free and open source distributed version control system. To the right is a diagram of a distributed version control system. Below the main text are four sections: About, Documentation, Downloads, and Community. The Downloads section is highlighted with an orange box and a callout bubble from the left that says "Download clients". To the right of these sections is a monitor displaying the latest source release 2.38.1 and a button to download for Windows. At the bottom, there are links for Pro Git, Windows GUIs, and Tarballs.

git --distributed-even-if-your-workflow-isnt

Git is a **free and open source** distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

Git is **easy to learn** and has a **tiny footprint with lightning fast performance**. It outclasses SCM tools like Subversion, CVS, Perforce, and ClearCase with features like **cheap local branching**, convenient **staging areas**, and **multiple workflows**.

About
The advantages of Git compared to other source control systems.

Documentation
Command reference pages, Pro Git book content, videos and other material.

Downloads
GUI clients and binary releases for all major platforms.

Community
Get involved! Bug reporting, mailing list, chat, development and more.

Latest source Release
2.38.1
Release Notes (2022-10-07)
Download for Windows

Pro Git by Scott Chacon and Ben Straub is available to read online for

Windows GUIs Tarballs

I use Git Bash in my examples

Create a New Repo in GitHub



- <https://github.com/>

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Owner * / Repository name *

Great repository names are short and memorable. Need inspiration? How about [silver-eureka?](#)

Description (optional)

☒ ☐ Public
Anyone on the internet can see this repository. You choose who can commit.

☐ Private
You choose who can see and commit to this repository.

Initialize this repository with:
Skip this step if you're importing an existing repository.

☐ Add a README file
This is where you can write a long description for your project. [Learn more.](#)

Add .gitignore
Choose which files not to track from a list of templates. [Learn more.](#)

Choose a license
A license tells others what they can and can't do with your code. [Learn more.](#)

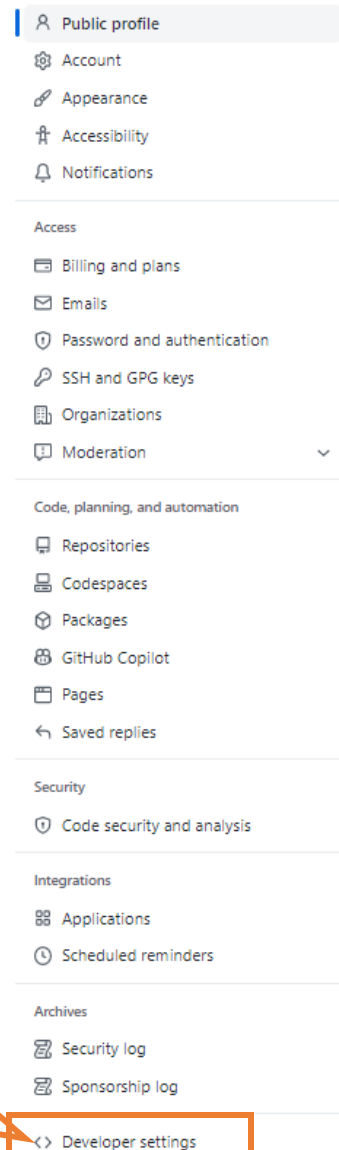
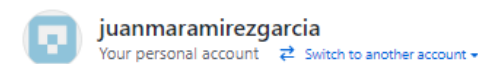
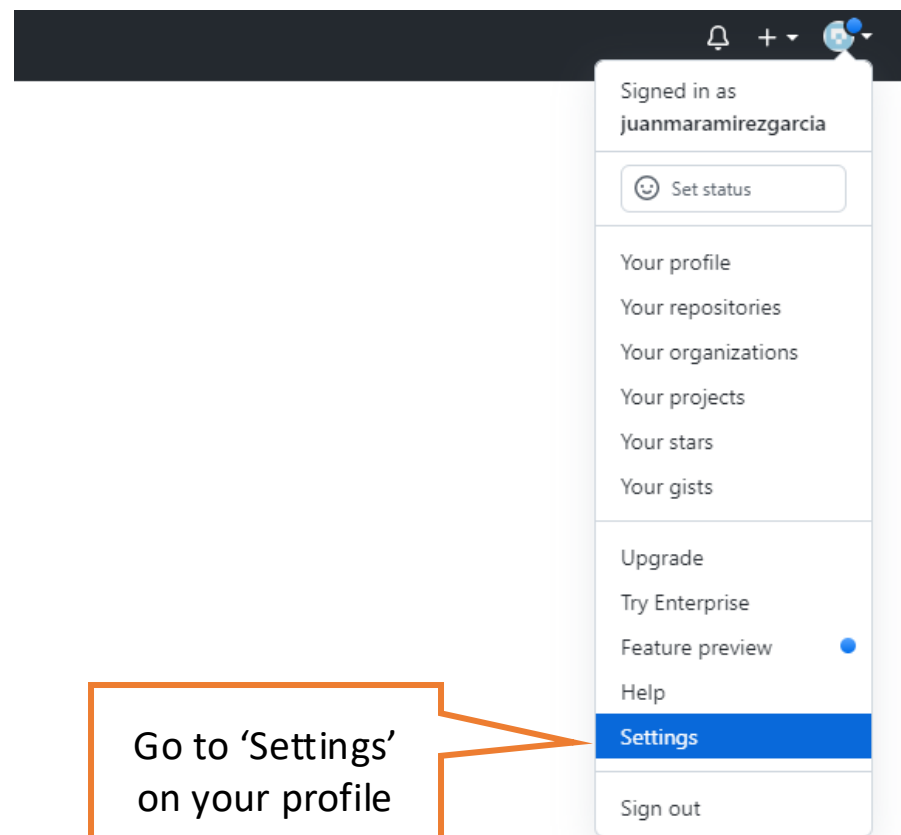
 You are creating a public repository in your personal account.

Give a name and
a description

Select 'Public' or
'Private'

Create repository

Generate Personal Access Token



Go to 'Developer settings'

Public profile

Name

Your name may appear around GitHub where you contribute or are mentioned. You can remove it at any time.

Public email

You have set your email address to private. To toggle email privacy, go to [email settings](#) and uncheck "Keep my email address private."

Bio

You can @mention other users and organizations to link to them.

URL

Twitter username

Company

You can @mention your company's GitHub organization to link it.

Location

☐ Display current local time

Other users will see the time difference from their local time.

All of the fields on this page are optional and can be deleted at any time, and by filling them out, you're giving us consent to share this data wherever your user profile appears. Please see our [privacy statement](#) to learn more about how we use this information.

[Update profile](#)

Contributions & Activity

Generate Personal Access Token (cont.)

Settings / Developer settings

- GitHub Apps
- OAuth Apps
- Personal access tokens
 - Fine-grained tokens Beta
 - Tokens (classic)

Personal access tokens (classic)

Generate new token ▾

Revoke all

Tokens you have generated that can be used to access the [GitHub API](#).

FUSE — repo

Expired on Sat, Jul 2 2022.

Last used within the last 6 months

Delete

My bash script — admin:repo_hook, delete_repo, project, repo

Expires on Wed, Jan 11 2023.

Last used within the last 6 months

Delete

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Click on
'Generate new
token'

Give the corresponding permissions (to interact with repositories) and don't forget to copy the token because if you lose it you will need to regenerate it again